Brachistochrone Optimization

Oscar Hernandez[1], John Rhodes
oihernandez@alaska.edu , j.rhodes@alaska.edu

November 27, 2020

Abstract

Consider a particle initially at rest which moves along a path according to the sole influence of gravity. We will confirm the first part and refute the second part of Galileo Galilei's early sixteenth-century conjecture, stated below.

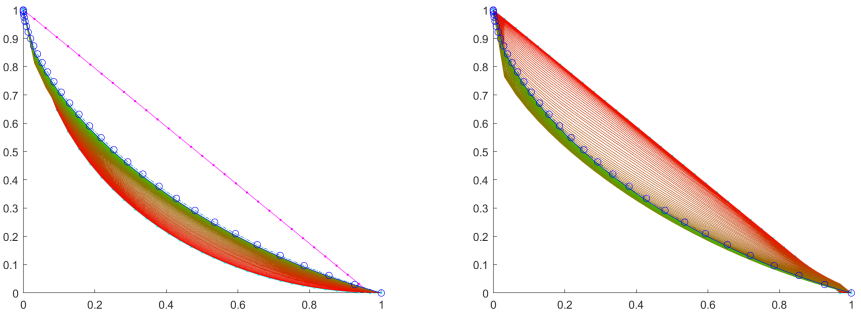
Conjecture 1 (Galileo, 1638)**.** *"It is possible to infer that the quickest path of all, from one point to another, is not the shortest path, namely, a straight line, but the arc of a circle."*

1 Introduction

Later in the sixteenth century, Johann Bernoulli officially posed the following *brachistochrone "least-time" problem* as an official 6-month challenge to "the most brilliant mathematicians in the world."

Problem 1 (Brachistochrone, 1696)**.** *"Given two points A and B in the vertical plane, what is the curve traced out by a point acted on only by gravity, which starts at A and reaches B in the shortest time?"*

We will show that there is a unique solution (green) which is neither a straight line segment nor the arc of a circle, and furthermore that it is possible to vary a *reasonably close* guess (red) to produce successively better approximations as shown in Figure 1 below.



(a) Red: Arc of a circle (b) Red: Straight line

Figure 1: Iteratively transform initial guesses (red) into optimal solution (green)

1.1 Overview

We will formalize the problem in Section 2, derive an analytic solution in Section 3 and discuss a computational implementation in Section 4.

[1]This paper serves as the first author's project submission to the second author's Fall 2020 course on (MATH F661) Optimization

2 Problem

After extending the deadline for a year and half, Bernoulli published his solution along with the submissions he received from his brother Jakob Bernoulli, Gottfried Leibniz, Ehrenfried Walther von Tschirnhaus, and an anonymous submission from Sir Isaac Newton. The original solutions included pure and differential geometric arguments as well as physical arguments from optics and from classical mechanics. Much of the history and problem formulation can be found in [1].

We will highlight a modern formulation of an argument from mathematical physics. Consider a particle with mass m whose vertical displacement is given by $y : I \to M$ where $I = [p_0, p_1]$, so that it travels along a curve $(x, y(x)) \subset M = \mathbb{R}^2$ from the starting point p_0 to the endpoint p_1.

Recall the following conservation law of kinetic energy and gravitational potential energy:

Law 1.

$$\frac{1}{2}mv^2 = mgy, \qquad v = \sqrt{2gy} \tag{1}$$

where g is the gravitational constant of acceleration.

Also, recall the line element which measures infinitesimal distances in the Euclidean plane:

Definition 1.

$$ds = \sqrt{dx^2 + dy^2} = \sqrt{dx^2 \left(\left(\frac{dx}{dx} \right)^2 + \left(\frac{dy}{dx} \right)^2 \right)} = \sqrt{1 + y'^2}\ dx. \tag{2}$$

Then, the total time traveled is given by the integral of infinitesimal $\frac{\text{distance}}{\text{distance/time}}$ as shown below:

$$t_{01} = \int_{p_0}^{p_1} \frac{ds}{v} = \int_{p_0}^{p_1} \frac{\sqrt{1 + y'^2}}{\sqrt{2gy}}\ dx \tag{3}$$

Thus, we can state the brachistochrone problem as the following optimization problem.

Problem 2 (Brachistochrone)**.** *Find the following minimizer:*

$$\min_{y(x)}\ \frac{1}{\sqrt{2g}} \int_{p_0}^{p_1} \sqrt{\frac{1 + y'(x)^2}{y(x)}}\ dx. \tag{4}$$

This may seem tractable for an experienced student of calculus, but it was not known at the time how to take the derivative of a function. Nevertheless, Leibniz and Newton independently managed to solve the problem the day each they were presented with it. In particular, Newton's solution confirmed the power of the calculus of variations which he had invented a decade earlier for the related problem of minimal resistance.

3 Analysis

Following Newton's calculus of variations approach, Leonhard Euler considered the more general problem below.

Problem 3 (Euler, 1740s)**.** *Given f, find the following minimizer $y(x)$:*

$$\min_{y(x)}\ \int_{p_0}^{p_1} f(x, y(x), y'(x))\ dx. \tag{5}$$

Notice that this reduces to our problem when we specify the integrand $f = f_0$ as below:

$$f_0(x, y(x), y'(x)) = \sqrt{\frac{1 + y'(x)^2}{2g \cdot y(x)}}.$$

This problem differs from others studied in this course in that we are asked to find a minimizing curve instead of a finite-dimensional vector.

We could transform this to such a problem by searching for the finite-dimensional vector $v^n = (v_0, v_1, \ldots, v_n)$ whose i^{th} entry corresponds to something close to $v_i = x\left(t_0 + i \cdot \frac{t_1 - t_0}{n}\right)$, and study limits of v^n as n tends to $+\infty$. We revisit this approach in Section 4.

Lagrange followed Euler's work and rephrased the problem as a search for a single constant as shown below:

$$\min_{\varepsilon \in \mathbb{R}} \int_{p_0}^{p_1} f(x, y(x) + \varepsilon \cdot z(x), y'(x) + \varepsilon \cdot z'(x))\, dt, \quad z \in C([p_0, p_1)] \text{ such that } z(p_0) = z(p_1) = 0, \tag{6}$$

where the constraints on z indicate that the arguments are perturbations joining the endpoints as desired. It follows that $y(x)$ is a solution to our problem iff $\varepsilon = 0$ is a solution to this problem, an optimality condition which would become the basis for his future work on Lagrange multipliers This line of research was later refined by Weierstrass in the 1870s, by Karush in the 1930s, and popularized by Kuhn and Tucker in the 1950s.

Setting the derivative of the integral equal to 0 and evaluating at $\varepsilon = 0$ yields the following optimality condition:

$$\frac{d}{dx}\frac{df}{dy'} - \frac{df}{dy} = 0,$$

We can solve this particularly nice Euler-Lagrange differential equation for y when given f. Since $\frac{\partial f}{\partial x} = 0$, we can apply the Beltrami identity $f - y' \frac{\partial f}{\partial y'} = C$ and compute $\frac{\partial f}{\partial y'} = y'(1+y'^2)^{-1/2}(2gy)^{-1/2}$, and obtain the following by subtraction:

$$\left(\frac{1}{\sqrt{2gy}\sqrt{1 + y'^2}}\right)^2 \cdot y\frac{(1 + y'^2)}{C^2} - 1 = C^2 \cdot y\frac{1 + y'^2}{C^2} - 1$$

$$y\left[1 + y'^2\right] = k^2, \qquad k = \frac{1}{2gC^2},$$

This differential equation admits the following parametric solution:

$$x = \frac{k^2}{2}(\theta - \sin\theta),$$
$$y = \frac{k^2}{2}(1 - \cos\theta),$$

which define the cycloid, whose shape is optimal. However, the constant of integration C and the parameter space of angles θ are not known and are not analytically derivable. Many classical mathematicians stop here, having found the optimal shape, confident in their abilities to transform the parametric curve otherwise. As applied mathematicians, we go on to solve two nonlinear equations for those two unknown parameters when given a choice of points p_0 and p_1 in Section 4.

4 Computation

We decompose the problem as Euler did.

$$\begin{aligned}
\min_{y(x)} \int_{p_0}^{p_1} \sqrt{\frac{1+y'^2}{2gy}}\, dx &= \min_{y(x)} \lim_{N\to\infty} \sum_{i=0}^{N-1} \int_{x_i}^{x_{i+1}} \sqrt{\frac{1+y'(x)^2}{2gy(x)}}\, dx && \text{by Riemann integration} \\
&\approx \min_{y(x)} \sum_{i=0}^{N-1} \int_{x_i}^{x_{i+1}} \sqrt{\frac{dx^2+dy^2}{2g\cdot y(x)}} && \text{for large } N \\
&\approx \min_{y=(y_0,y_1,\ldots,y_{N-1})} \sum_{i=0}^{N-1} \sqrt{\frac{(x_{i+1}-x_i)^2+(y_{i+1}-y_i)^2}{2g\cdot \frac{y_{i+1}+y_i}{2}}} && \text{for } y_i = y(x_i)
\end{aligned}$$

We view this as an optimization problem with no specified constraints whose non-linear objective function is the partial sum above, and solve it through a custom implementation of the (quasi-Newton) Broyden–Fletcher–Goldfarb–Shanno algorithm described in detail below.

Code Walkthrough

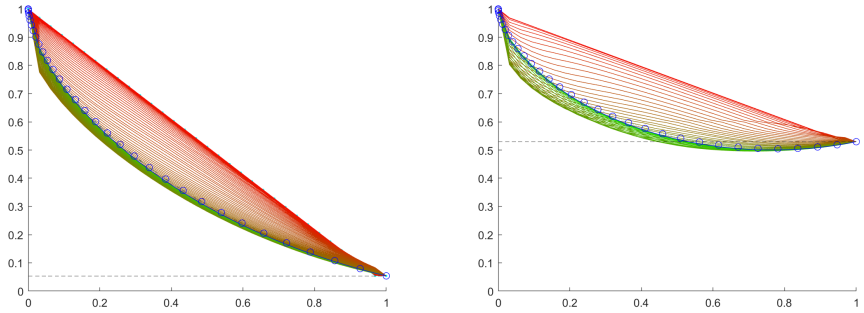
We discuss some aspects of the computer program attached as Appendix A We start by setting some hyper-parameters and "input" variables. Our main function `plot_solutions` consists of three parts:

1. calling our `gen_cycloid` method to identify the optimal cycloid's parameters
2. calling our `gen_cycloid method` to identify the optimal vector $y_\star$
3. plotting the intermediate vector steps and the final solutions.

As discussed in section 3, the solution to the optimality condition is a parametric curve defined whose geometry is defined by two parameters, the radius of the circle which traces the cycloid, and the total angle elapsed. The `gen_cycloid` methods uses MATLAB's symbolic solver; it indeed fails to derive an analytic solution but it succeeds in finding numerical estimates which agree with the optimal solution identified by the quasi-Newton method.

The BFGS implementation is based on a program OH developed in a previous homework assignment for this class. The novel contribution is the numerical integration which implements the midpoint formula. As is common, there are many ways to further "optimize" this computer program. We seek efficiency in computation, unless it interferes with the program's clarity or modularity. However, we prioritized clarity, modularity, and proof of correctness over epi-efficiency.

We now return to the intuition which inspired this problem. It may be surprising that the shortest path between two points (namely the straight line joining them) is not the one which admits least-time travel. But it's a good guess, so we used as it as our initial guess (as shown in red). Then, we travel in the newton direction drawing each new curve in a hue which is decreasingly red and increasingly green. Sometimes the curve stays above the final height, and sometimes it drops below, depending on the choice of endpoints.



(a) (b)

Figure 2

A MATLAB Code

```
1
2  % author: oihernandez@alaska.edu
3  % Quasi-Newton solution the brachistochrone problems (1 numerical and
4  % 2 pseudo-symbolic)
5
6  clc; clf; clear;
7  format long;
8
9  global max_iter m_iter delta c1 c2 N p0 p1 y0 yn;
10
11 % tolerances for QN and Wolfe conditions
12 max_iter = 10;
13 m_iter = floor(max_iter/2);
14 delta = 10^(-max_iter);
15 c1 = 10^(-4);    % used in Armijo condition
16 c2 = 0.9;        % used in crvature condition
17 assert(c1 < c2)
18
19 N = 32;         % number of terms in integral approximate sum
20 p0 = [0; 1];  % starting point
21 h = normpdf(0,2); % test photogenic height
22 p1 = [1; h]    % end point
23 assert(p0(2) > p1(2)); % so that the particle can slide down
24
25 plot_solutions()
26 % solve and plot
27 function plot_solutions(cyc)
28     global N m_iter p0 p1 y0 yn;
29     % solve in reference prame at p0 and pointing toward p1,
30     % so that the analytic solution is given by a standard cycloid.
31     hold on;
32     ax = [0 p1(1) min(0,p1(2)) p0(2)];
33     axis(ax);
```

```
34      yline(p1(2), '--');
35      cyc = gen_cycloid();
36      plot(cyc(1,:), cyc(2,:), '-.');
37      % reflect vertical displacements and then reflect solution back
38      y0 = 0;
39      yn = 1 - p1(2); % constrained by endpoints, but free to optimize
40      y = [1/N: (yn-1/N)/(N-2): yn]'; % intermediate entries.
41      plot(0:1/N:1, [p0(2);1-y;p1(2)],'.-', 'Color', [1,0,1]);
42  %     y = flipud(sin(acos(y))); % intermediate entries.
43  %     y(1) = y(1)+m_iter/N;
44      plot(0:1/N:1, [p0(2);1-y;p1(2)],'.-', 'Color', [0,1,1]);
45
46      drawnow;
47      x = 1 - qNewton(y, @time, @dtime);
48      r = size(x,2);
49      l = eval(time(cyc(2,:)))
50      % Plot/Animate
51      for k = 2:r
52          pause(2/r);
53  %         t = time(x(:,k));
54          plot(0:1/N:1, [p0(2);x(:,k);p1(2)],'-', 'Color', [1-k/r,k/r,0]);
55          axis(ax);
56      end
57      plot(0:1/N:1, [p0(2);x(:,r);p1(2)],'-', 'Color', [0,0,1]);
58      plot(cyc(1,:),cyc(2,:),'ro', 'Color', 'blue');
59  end
60
61  function cyc = gen_cycloid()
62      global N p1;
63      th0 = 0;    % cycloid's corresponding circle has rotated 0 degrees
            at start
64      syms th1 R; % find cycloid parameters by solving for its symbolic
            vars
65      [th1, R] = solve([p1(1)==R*(th1-sin(th1)), p1(2)==1-R*(1-cos(th1))],
            [th1 R]);
66      th = [th0:th1/N:th1];   % parameter space of angles
67      cyc = [R*(th-sin(th)); 1-R*(1-cos(th))]; % cycloid curve
68  end
69
70  function x = qNewton(xk, g, dg)
71      global max_iter m_iter delta c1 c2 N p0 p1 y0 yn;
72      k = 1;                  % current iteration
73      x(:,k) = xk;            % current estimate
74      d = dg(x(:,k));         % current gradient
75      assert(size(d,2)==1);   % is a vector
76      Bk = eye(size(d,1));        % current Hessian approximation
77      e = 1;                  % "current" error
78      while e>delta && k < 10^max_iter
79          z =  g(x(:,k));     % current value
80          p(:,k) = -Bk\d;     % Newton direction
81
```

```
82          while not(is_descent_direction(p(:,k),d))
83              p(:,k) = gradient_direction(d, 1);
84          end
85          ak = 10^(-m_iter); % steplength alpha
86          akc = 0;
87          ac = g(x(:,k)+ak*p(:,k)) <= z+c1*ak*p(:,k)'*d;
88          while not(ac && -p(:,k)'*dg(x(:,k)+ak*p(:,k)) <= -c2*p(:,k)'*d
                || isinf(ak))
89              ak = ak*(1+1/N);
90              akc= akc+1;
91              if akc== 10^(m_iter)
92                  return % because new estimates within margin of error.
93              end
94          end
95          if isnan(g(x(:,k)+ak*p(:,k)))
96              return % without adding Nan entry
97          end
98
99          s(:,k) = ak*p(:,k);
100         x(:,k+1)=x(:,k)+s(:,k);
101         y(:,k) = dg(x(:,k+1))-d;
102         Bk = BFGS_update(Bk, y(:,k), s(:,k));
103         k = k + 1;
104         z =  g(x(:,k));
105         d = dg(x(:,k));
106         e = norm(d)/(1+norm(d));
107     end
108 end
109
110 function Bkp = BFGS_update(Bk, yk, sk)
111     Bkp = Bk + (yk*yk')/(yk'*sk) - (Bk*sk*sk'*Bk)/(sk'*Bk*sk);
112 end
113
114 function gd = gradient_direction(d, alpha)
115     gd = -alpha*d;
116 end
117
118 function b = is_descent_direction(p, dfxk)
119     b = p.'*dfxk < 0;
120 end
121
122 function d = dtime(y)
123     global m_iter
124     n = size(y, 1) + 1;
125     d = zeros(n-1, 1);
126     for i = 1:n-1
127         dy = time([y(1:i-1);y(i)*(1+0.5/n);y(i+1:n-1)]) -  time([y(1:i
                -1);y(i)*(1+-0.5/n);y(i+1:n-1)]);
128         d(i) = dy/(1/n);
129     end
130 end
```

```
131
132 function t01 = time(y) % riemann sum of f0 from y0 to yn
133     global y0 yn;
134     n = size(y, 1) + 1;
135     t01 = f0(n,y0,y(1)) + f0(n,y(n-1),yn);
136     for i = 2 : n-1 % outer terms + inner terms
137         t01 = t01 + f0(n,y(i-1),y(i));
138     end
139 end
140
141 function fxk = f0(n, yk,ykp) % time elapsed during infinetismal travel
142     fxk = ds(1/n, ykp - yk) / v(yk, ykp); % time = distance/(distance/
        time)
143 end
144
145 function dist = ds(dx, dy) % Euclidean line element
146     dist = sqrt(dx^2+dy^2);
147 end
148
149 function vel = v(yk, ykp) % midpoint/average velocity from time k to kp.
150     grav = 9.80665; % constant of gravitational acceleration in m/s^2
151     vel = (sqrt(2*grav*yk) + sqrt(2*grav*ykp))/2; % terms come from
        conservation law
152 end
```

References

[1] I. Griva, S. Nash, and A. Sofer, *Linear and Nonlinear Optimization: Second Edition*, ser. Other Titles in Applied Mathematics. Society for Industrial and Applied Mathematics (SIAM, 3600 Market Street, Floor 6, Philadelphia, PA 19104), 2009. [Online]. Available: https://books.google.com/books?id=uOJ-Vg1BnKgC